

Array Operations

Learning Objectives

- Perform common array operations: insert, delete, search, and sort
- Analyze the efficiency of different array operations
- Implement algorithms that manipulate arrays to solve problems

Engage (5-7 minutes)

A library has hundreds of books arranged on shelves. Finding a specific book, adding a new one, or rearranging them alphabetically are all operations we perform on collections. Arrays work the same way in programming—you need to know how to insert new data, remove old data, search for specific items, and sort them. Today we master these essential array operations.

Explore (10-12 minutes)

Start with the array [45, 12, 78, 34, 90, 23]. Try these operations manually on paper: insert 56 at position 2, delete the element at position 4, search for 78, and sort the array in ascending order. Count how many steps each operation takes. Now compare: is searching faster when the array is sorted or unsorted?

Explain (10-12 minutes)

Array operations include insertion (adding an element at a specific position), deletion (removing an element), search (finding an element's position), and sorting (arranging elements in order). Insertion and deletion in the middle of an array require shifting elements, making them $O(n)$ operations. Linear search checks each element sequentially at $O(n)$, while binary search on a sorted array achieves $O(\log n)$. Sorting algorithms vary from $O(n^2)$ like bubble sort to $O(n \log n)$ like merge sort.

Elaborate (8-10 minutes)

Implement a bubble sort algorithm to sort an array of 15 random numbers. Count the number of comparisons and swaps. Then implement a linear search and a binary search on the sorted array, comparing how many steps each takes. Write a function that removes all duplicate values from an array. Discuss which operations would be inefficient on very large arrays and why.

Evaluate (5-8 minutes)

Given the array [5, 2, 8, 1, 9, 3], trace through bubble sort step by step, showing the array state after each pass. Then perform binary search for the value 8 on the sorted result and count the comparisons. Explain why binary search requires a sorted array.